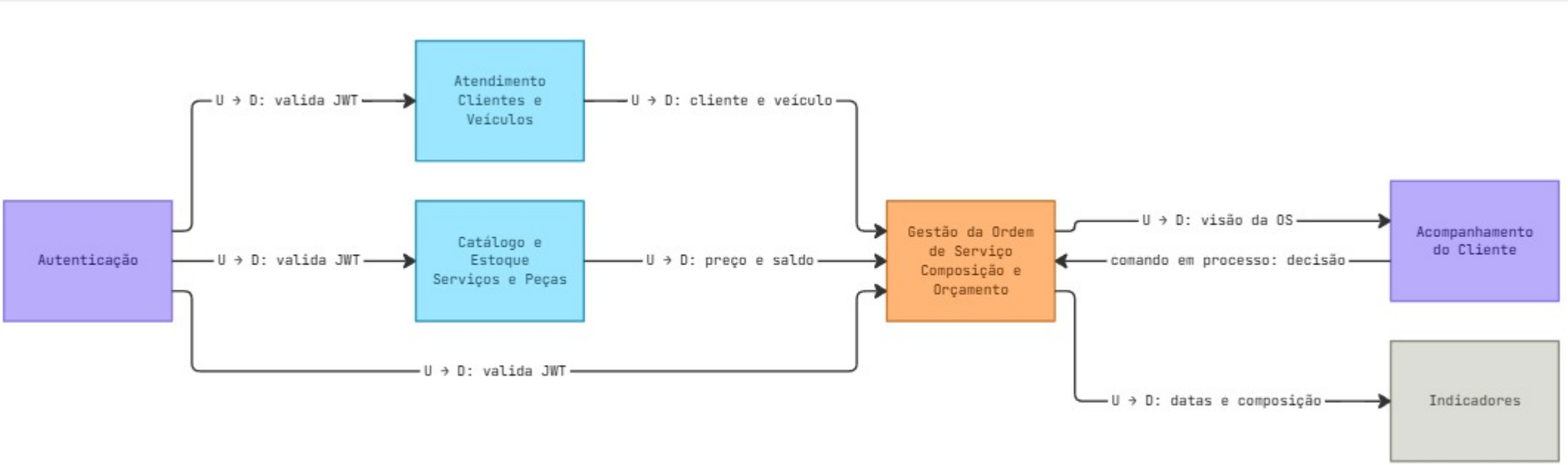
Bounded Contexts e relações entre módulos



U = upstream; D = downstream. Separação lógica em src/modulos, com comunicação no mesmo processo e persistência PostgreSQL compartilhada.

Responsabilidades

Gestão da Ordem de Serviço — Core Domain
Núcleo do ciclo da OS, composição, orçamento e transições.

Atendimento — Supporting / upstream
Clientes e veículos fornecem identidades validadas para a OS downstream.

Catálogo e Estoque — Supporting / upstream
Fornece serviços, preços e saldo; a OS coordena a baixa transacional.

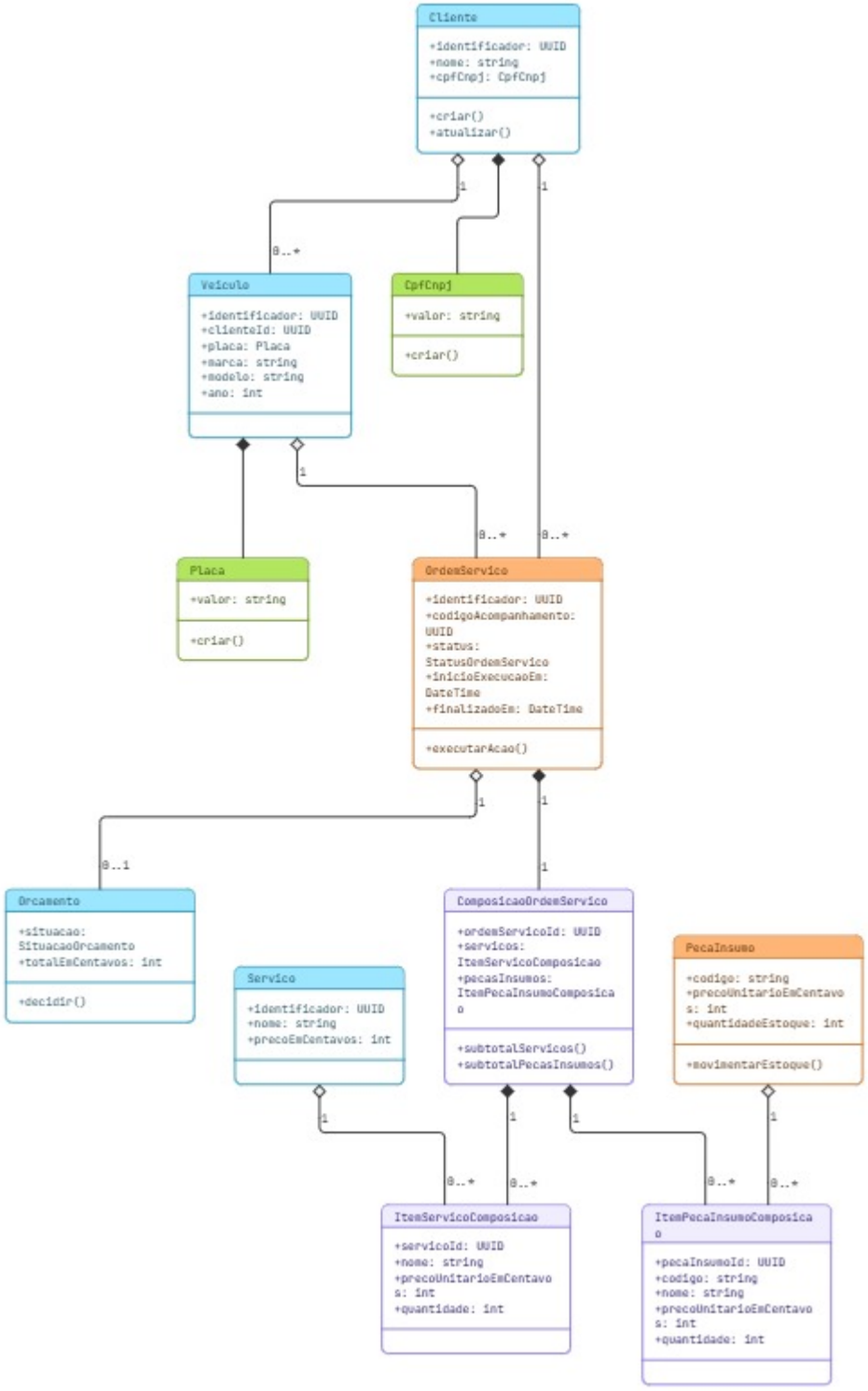
Acompanhamento — Supporting / downstream
Consome a visão da OS e encaminha a decisão autenticada pelo código privado.

Indicadores — Generic / downstream
Calcula médias a partir das datas e da composição persistidas pela OS.

Autenticação — Generic / upstream transversal
Emite e valida JWT consumido pelas APIs administrativas.

Integração real
Comunicação em processo por chamadas de casos de uso, contratos TypeScript e adaptadores Prisma. Não há mensageria, API externa ou microsserviços.

Modelo de domínio — entidades, objetos de valor e relacionamentos



Os itens da composição armazenam nome, código e preço unitário para preservar o orçamento da OS mesmo após alterações futuras no catálogo.

Classificação DDD

Entidades e raízes

• OrdemServico: raiz do agregado central e ciclo de vida
• Orcamento: entidade ligada 1:1 à OS, com decisão única
• PecaInsumo: raiz responsável pela consistência do estoque
• Cliente, Veiculo e Servico: entidades/raízes cadastrais com repositórios e casos de uso próprios

Componente e snapshots

• ComposicaoOrdemServico: componente associado à OS
• ItemServicoComposicao e ItemPecaInsumoComposicao: interfaces/snapshots; no Prisma são ItemServicoOrdem e ItemPecaInsumoOrdem

Objetos de valor implementados

• CpfCnpj: valida e normaliza CPF ou CNPJ
• Placa: valida padrão antigo e Mercosul

Enums e tipos do domínio

• StatusOrdemServico e AcaoOrdemServico
• DecisaoOrcamento e SituacaoOrcamento
• TipoMovimentacaoEstoque

Invariantes

Campos obrigatórios; centavos inteiros; estoque não negativo; transições válidas; orçamento decidido uma única vez.



Efeitos das transições

INICIAR_DIAGNOSTICO
RECEBIDA → EM_DIAGNOSTICO. Libera a definição dos serviços e peças necessários.

SOLICITAR_APROVACAO
EM_DIAGNOSTICO → AGUARDANDO_APROVACAO. Gera o orçamento e o deixa como PENDENTE.

APROVAR
AGUARDANDO_APROVACAO → EM_EXECUCAO. Registra o início da execução e baixa as peças do estoque.

REJEITAR
AGUARDANDO_APROVACAO → EM_DIAGNOSTICO. Permite revisar a composição sem alterar o estoque.

FINALIZAR
EM_EXECUCAO → FINALIZADA. Registra a data de término do serviço.

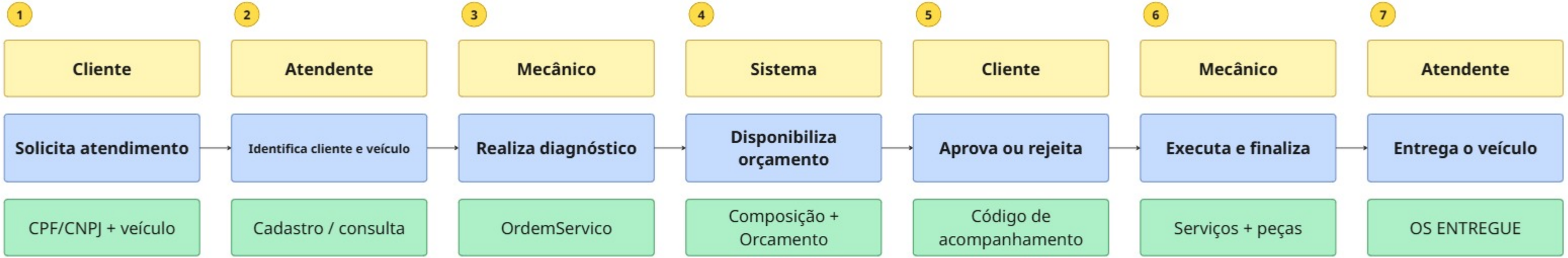
ENTREGAR
FINALIZADA → ENTREGUE. Encerra o ciclo da Ordem de Serviço.

Qualquer ação executada fora do status esperado é rejeitada pelo domínio.

A rota genérica de ações executa início de diagnóstico, finalização e entrega. Solicitação, aprovação e rejeição passam pelos casos de uso de orçamento para garantir consistência entre OS, orçamento e estoque.

Domain Storytelling: atendimento completo

Cenário principal sem desvios: da solicitação do cliente à entrega do veículo.



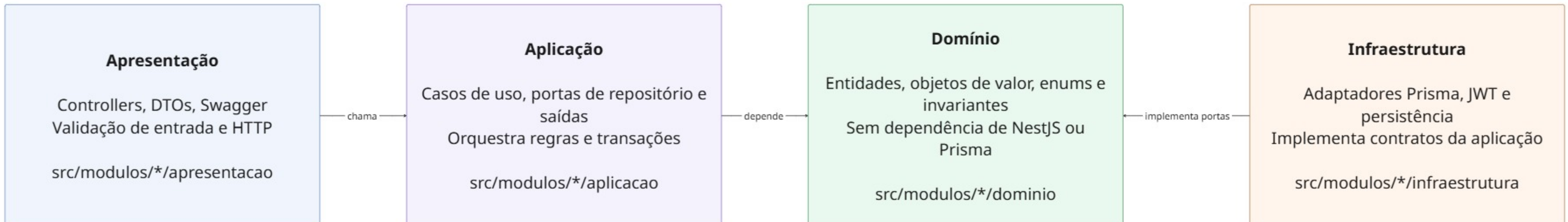
Grupo: atendimento e diagnóstico
Atores: Cliente, Atendente e Mecânico
Objetos: cadastro, veículo e OrdemServico

Grupo: decisão, execução e entrega
Atores: Cliente, Mecânico, Atendente e Sistema
Objetos: composição, orçamento, estoque e status

Anotação: o Sistema representa automações do monólito

Arquitetura DDD em camadas

Dependências apontam para aplicação e domínio; adaptadores Prisma implementam contratos internos.



Monólito modular NestJS: os contextos lógicos executam no mesmo processo. A separação em camadas organiza responsabilidades, mas não representa microsserviços nem isolamento físico.